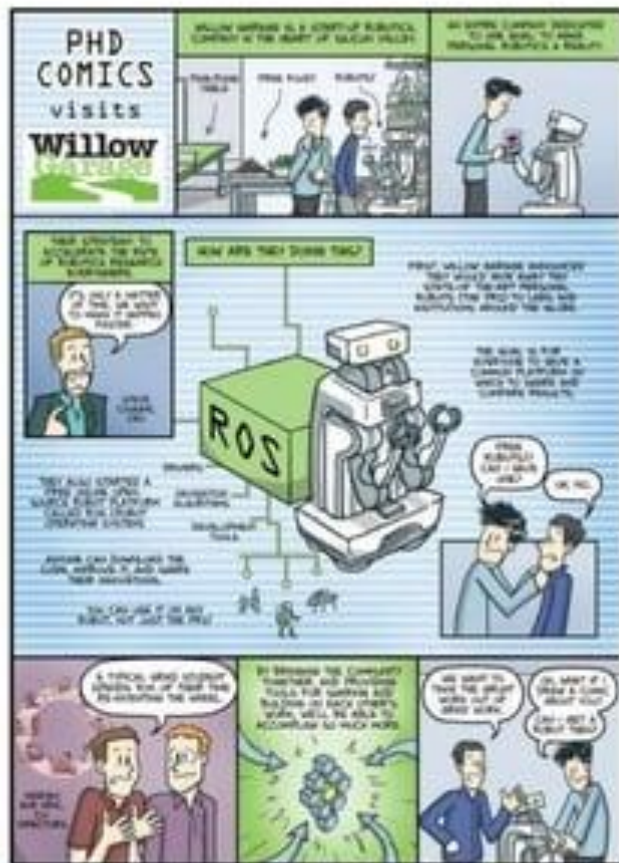


What is ROS?

- *ROS is an open-source, meta-operating system for your robot. It provides the services you would expect from an operating system, including hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management.*
- *It also provides tools and libraries for obtaining, building, writing, and running code across multiple computers.*
- *ROS is similar in some respects to 'robot frameworks, such as Player, YARP, Orocos, CARMEN, Orca, MOOS, and Microsoft Robotics Studio.*



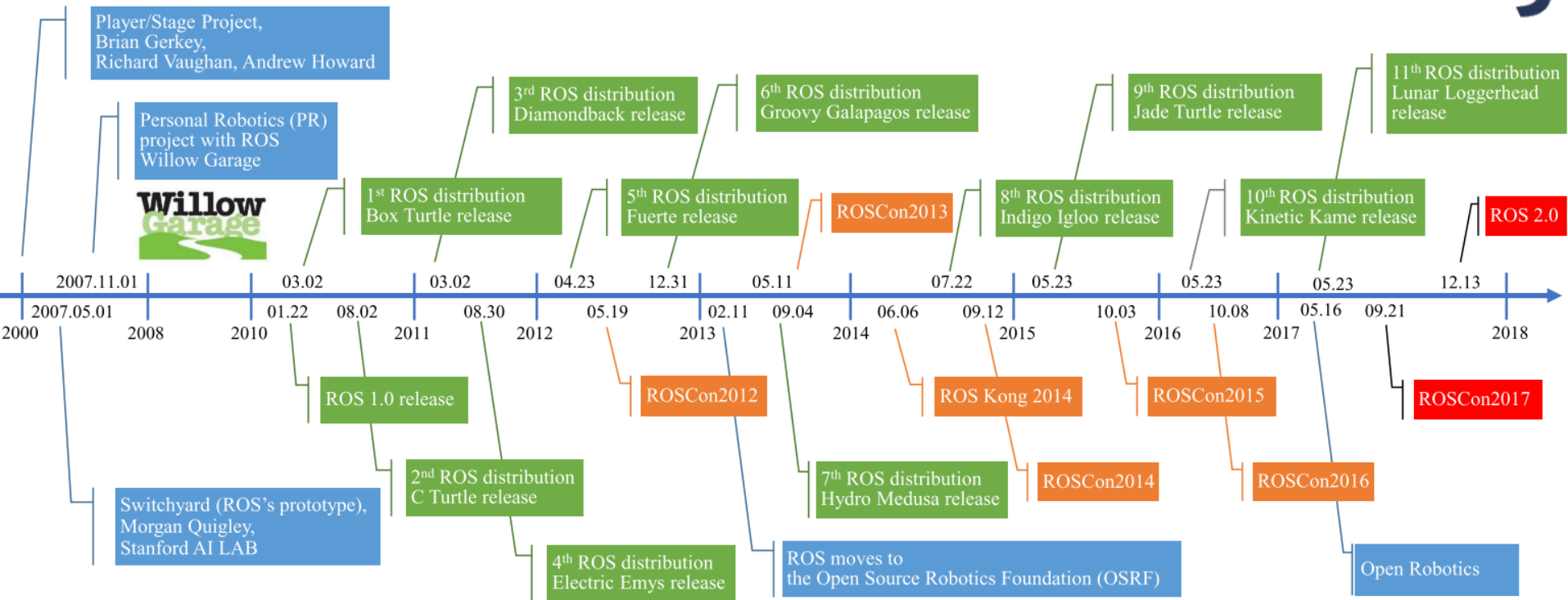
The ROS Equation

The ROS project can be defined in a single equation

- **1. Plumbing:** ROS provides publish-subscribe messaging infrastructure designed to support the quick and easy construction of distributed computing systems.
- **2. Tools:** ROS provides an extensive set of tools for configuring, starting, introspecting, debugging, visualizing, logging, testing, and stopping distributed computing systems.
- **3. Capabilities:** ROS provides a broad collection of libraries that implement useful robot functionality, with a focus on mobility, manipulation, and perception.
- **4. Ecosystem:** ROS is supported and improved by a large community, with a strong focus on integration and documentation. ros.org is a one-stop-shop for finding and learning about the thousands of ROS packages that are available from developers around the world.



ROS History



Open Source Robotics Foundation



open
robotics



ROS2



22.04



ubuntu



20.04



ubuntu

18.04

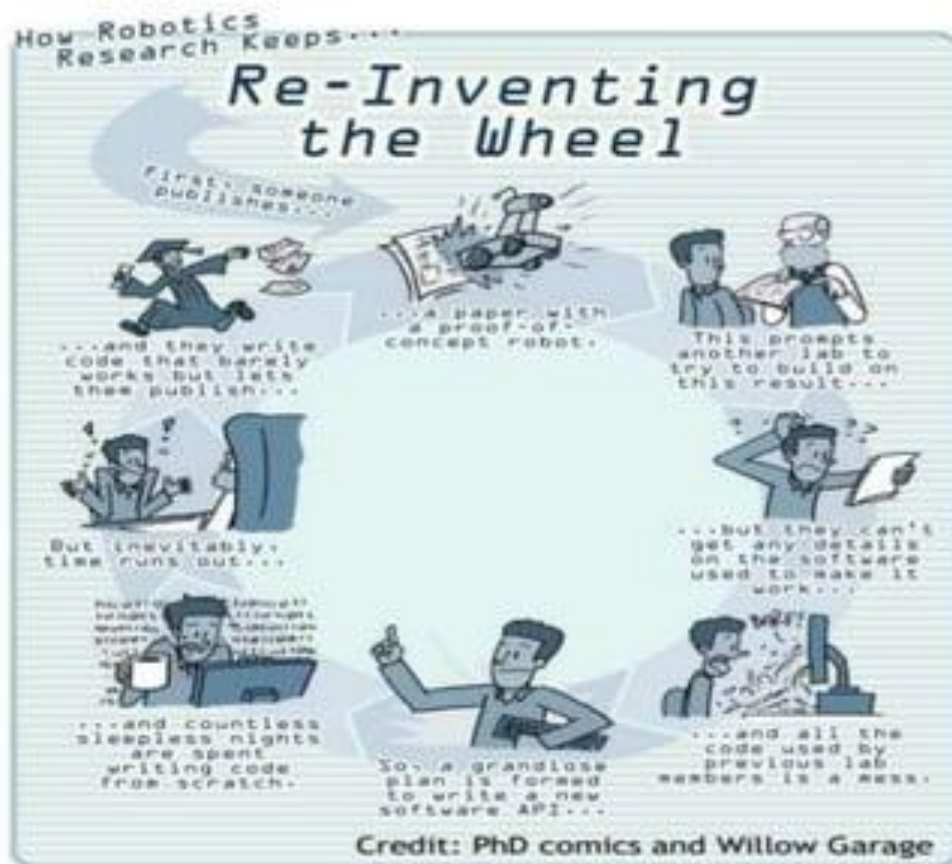


ubuntu



Why ROS?

- free and open source robotics software framework
- Message passing interface between processes
- Operating system-like features.
- High-level programming language support and tools.
- Availability of third-party libraries.
- Off-the-shelf algorithms.
- Ease in prototyping
- Ecosystem/community support.
- Extensive tools and simulators



ROS contains many open source implementations of common robotics functionality and algorithms. These open source implementations are organized into "packages". Many packages are included as part of ROS distributions, while others may be developed by individuals and distributed through code sharing sites such as github. Some packages of note include:



Motion planning



**Mapping and
localization**



Navigation



Perception



**Coordinate
frame
representation**



Simulation

ROS Robots

<https://robots.ros.org/>

ROS





When to use ROS?!

- **1- The robot contains many different sensors and actuators.** ROS will help you create a distributed system for all those components, instead of having one big monolith code base that will be hard to maintain and scale.
- **2- For Education Purposes.** ROS helps students see the full picture of robotics applications.
- **3- For Research Purposes.** Prevents re-inventing the wheel.
- **4- For Fast Prototyping**

Why might ROS be not preferable?!



Difficulty in learning:

ROS can be difficult to learn. It has a steep learning curve and developers should become familiar with many new concepts to get benefits from the ROS framework.



Difficulties in starting with simulation:

The toughness of learning simulation using Gazebo and ROS is a reason for not using it in projects.



Difficulties in robot modeling:

Learning to model a robot in ROS will take a lot of time, and building using URDF tags is also time-consuming compared to other simulators.



Potential limitations:

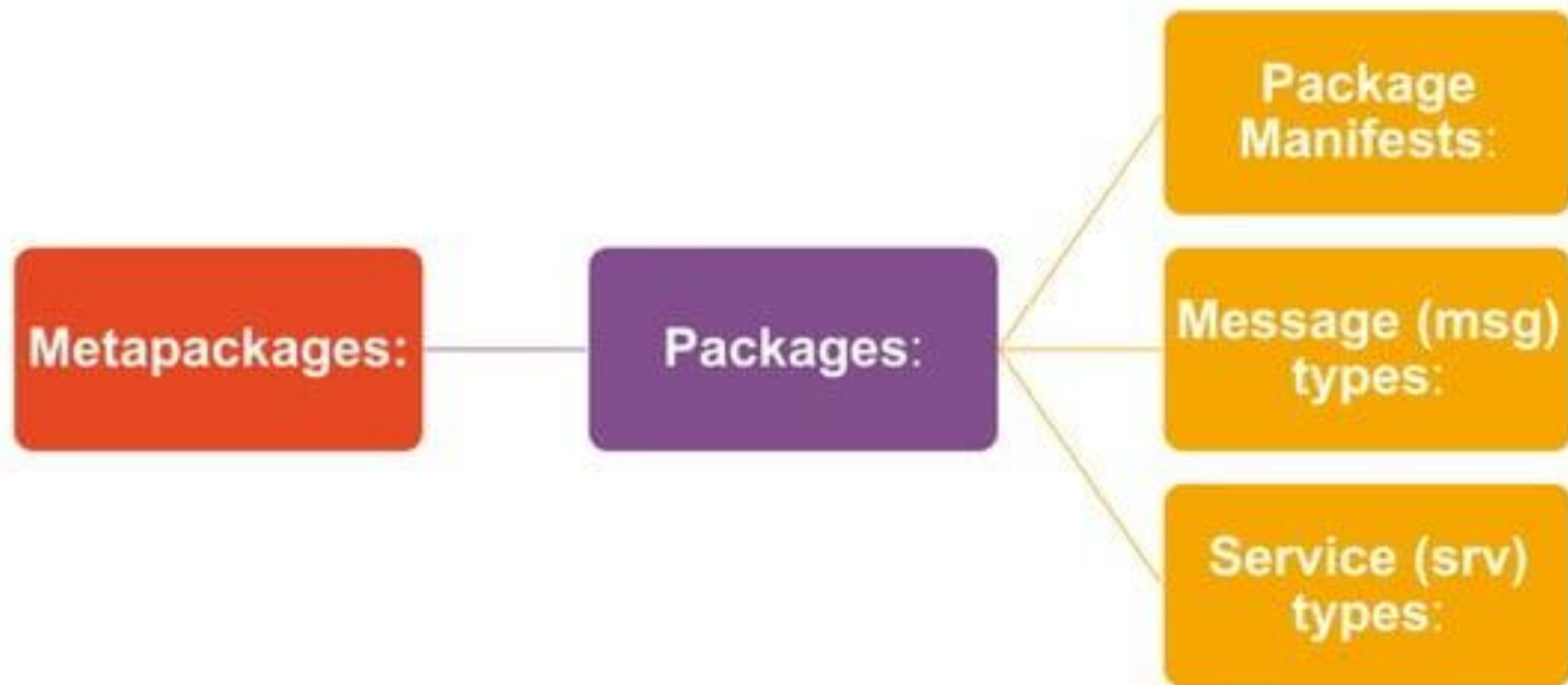
- 1- complexity to implement robust multi-robot distributed applications.
- 2- There is a lack of a native real-time application development support

ROS Concepts

There are mainly three levels of ROS:

- The ROS file system
- The ROS Computation Graph
- The ROS community

ROS filesystem



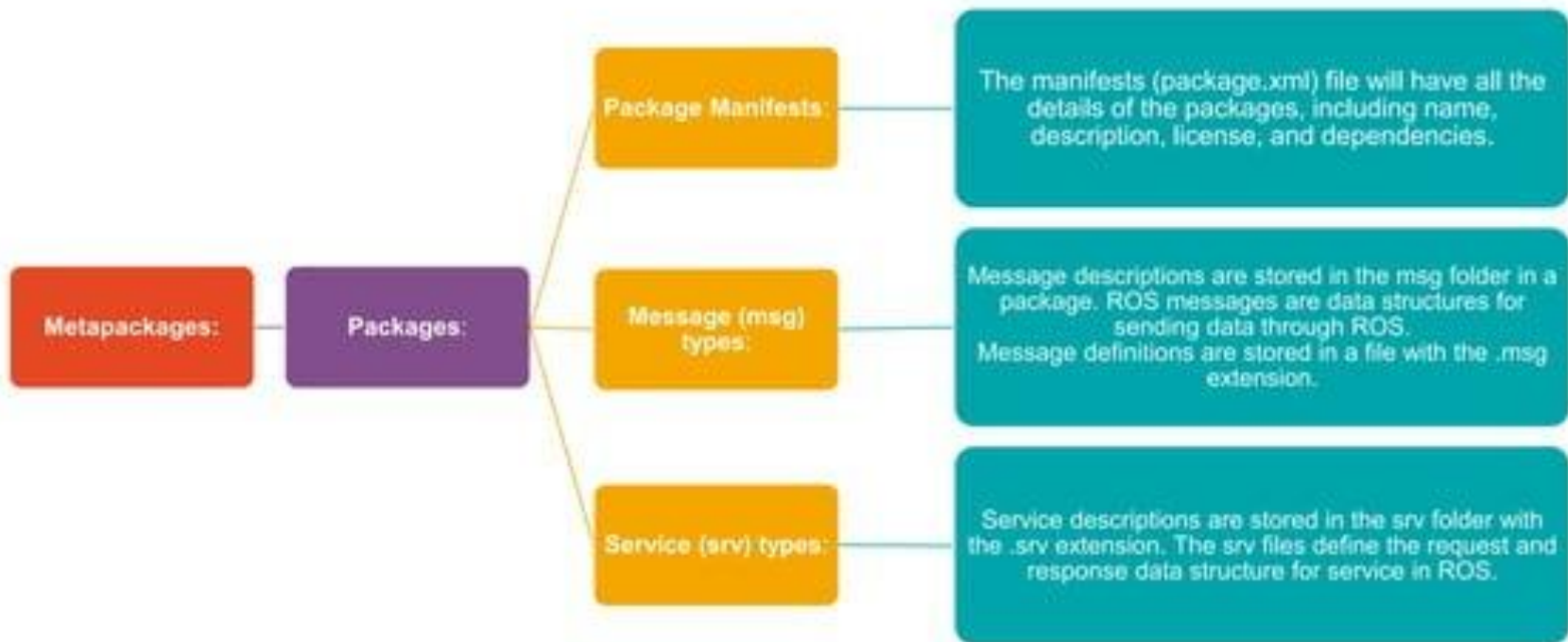
ROS filesystem

Metapackages:

Packages:

ROS packages are the main unit of an ROS software framework. A ROS package may contain executables, ROS-dependent library, configuration files, and so on. ROS packages can be reused and shared.

ROS filesystem

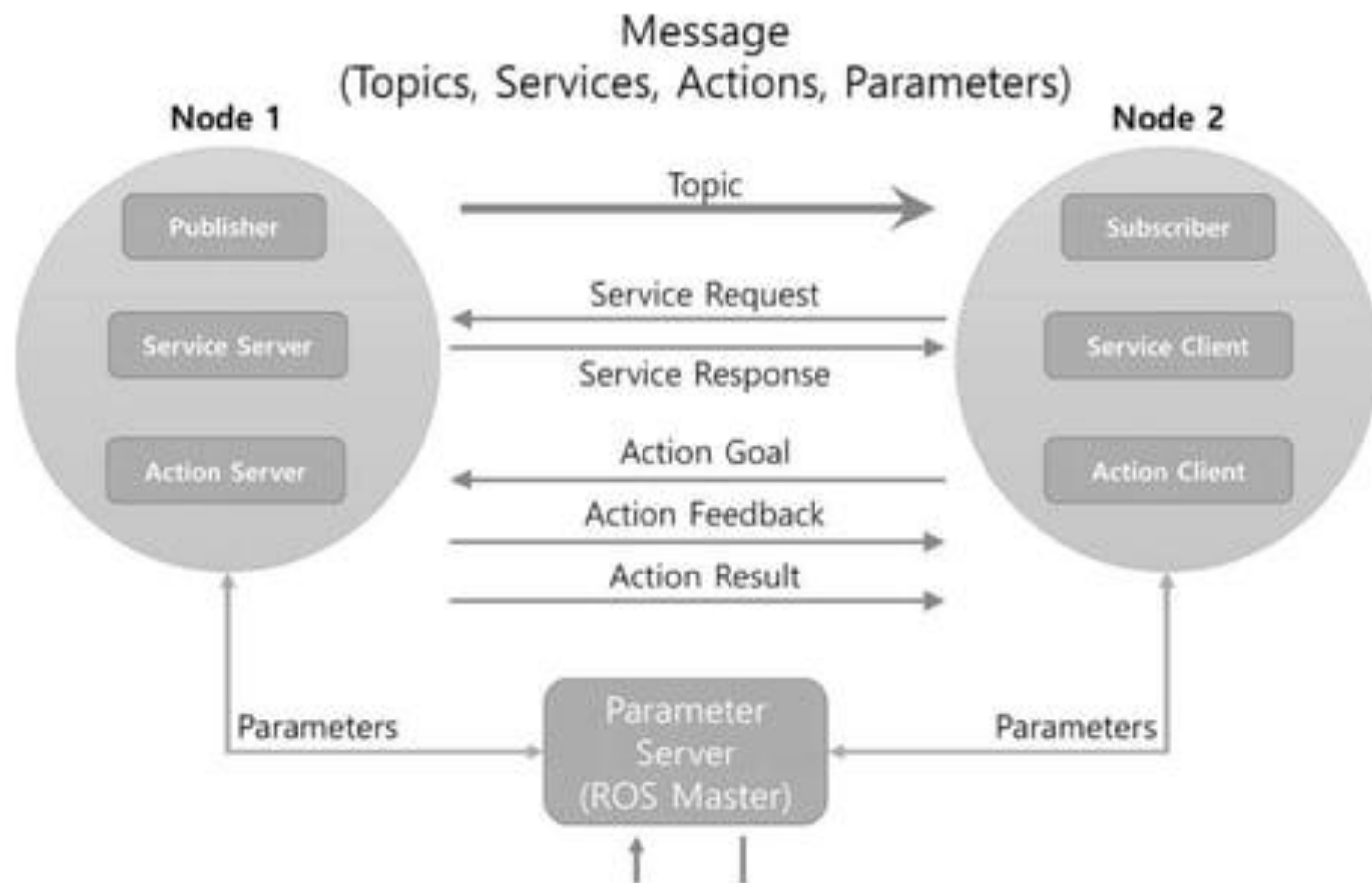


ROS Computation Graph Level

The ROS Computation Graph is the peer-to-peer network of the ROS process that processes data together.

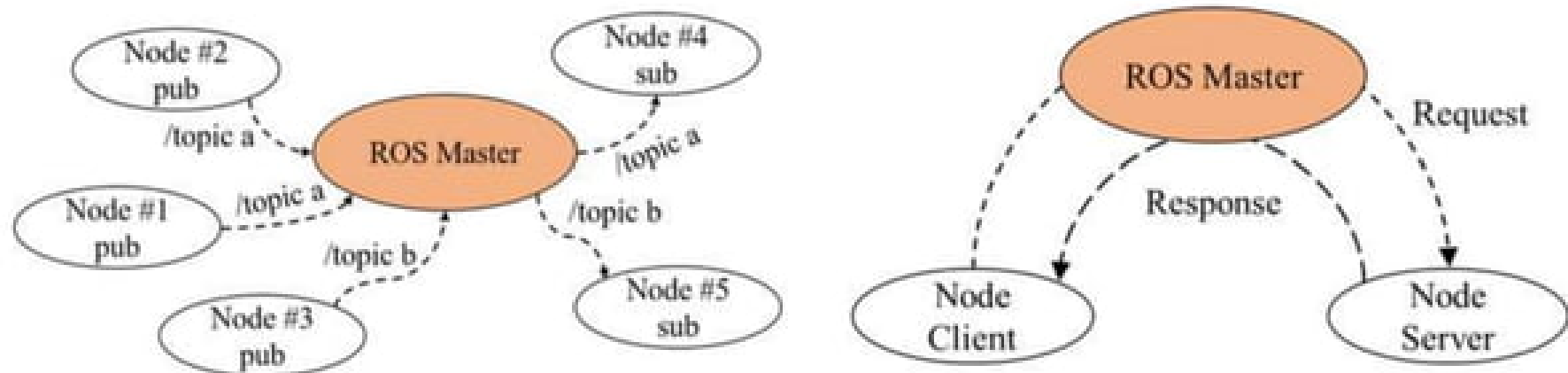
The following are the basic concepts of ROS CGL:





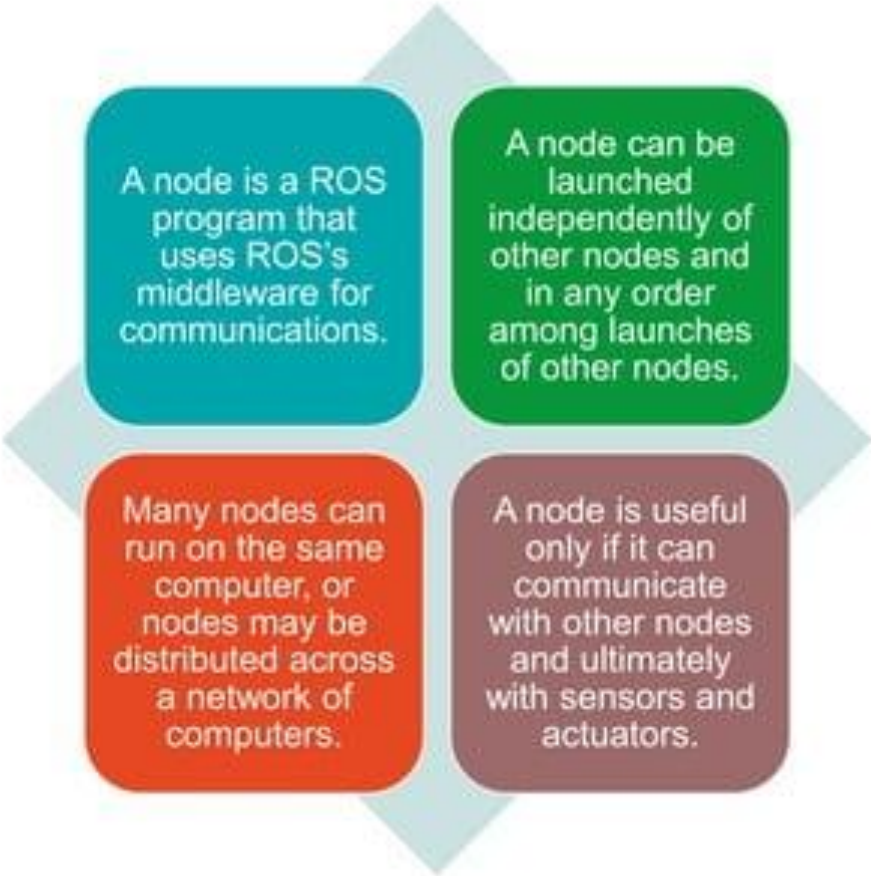
ROS Nodes

- A node really isn't much more than an executable file within a ROS package. ROS nodes use a ROS client library to communicate with other nodes. Nodes can publish or subscribe to a Topic. Nodes can also provide or use a Service.





ROS Nodes



A node is a ROS program that uses ROS's middleware for communications.

The diagram consists of a central light blue diamond shape. Inside this diamond are four rounded rectangular boxes, each containing a characteristic of ROS nodes. The boxes are colored teal (top-left), green (top-right), orange (bottom-left), and maroon (bottom-right). The text inside the boxes is white, except for the top-left box which has black text.

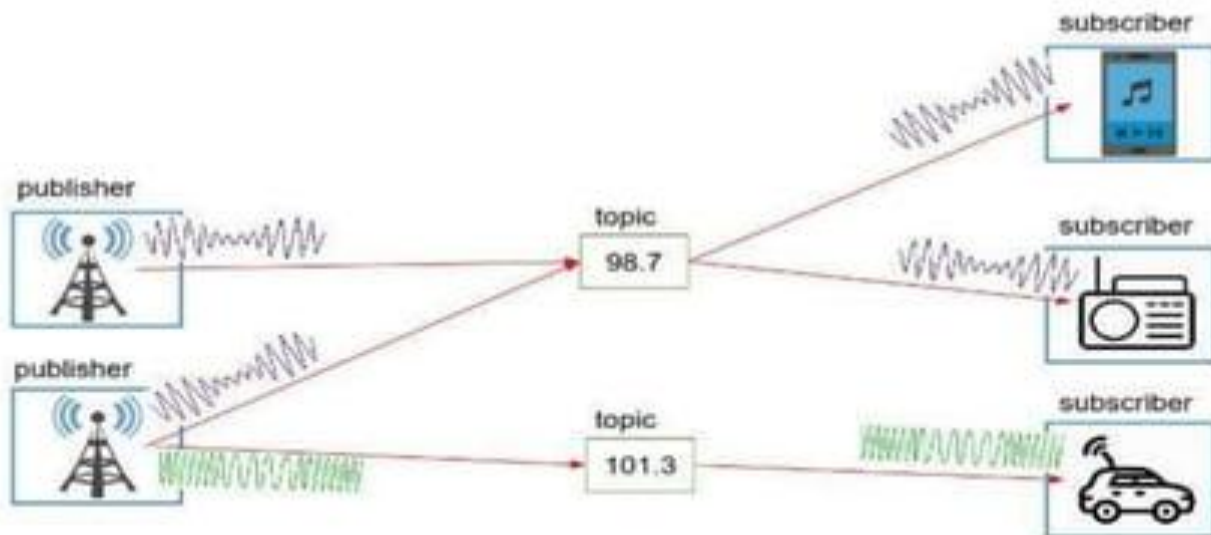
A node can be launched independently of other nodes and in any order among launches of other nodes.

Many nodes can run on the same computer, or nodes may be distributed across a network of computers.

A node is useful only if it can communicate with other nodes and ultimately with sensors and actuators.

ROS Topics

- *ROS topics are Named buses in which ROS nodes can send a message. A node can publish or subscribe any number of topics.*



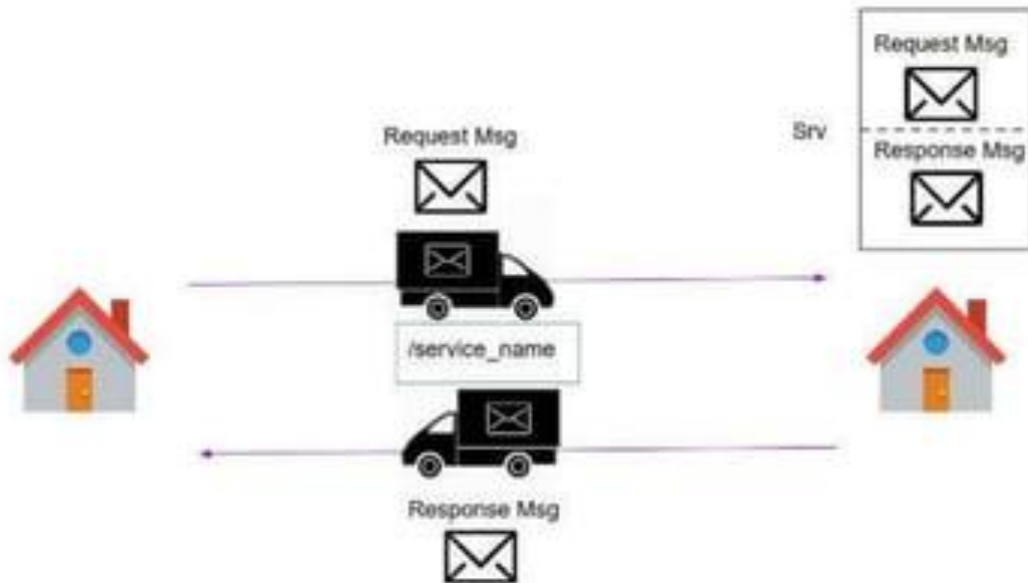


ROS Topics

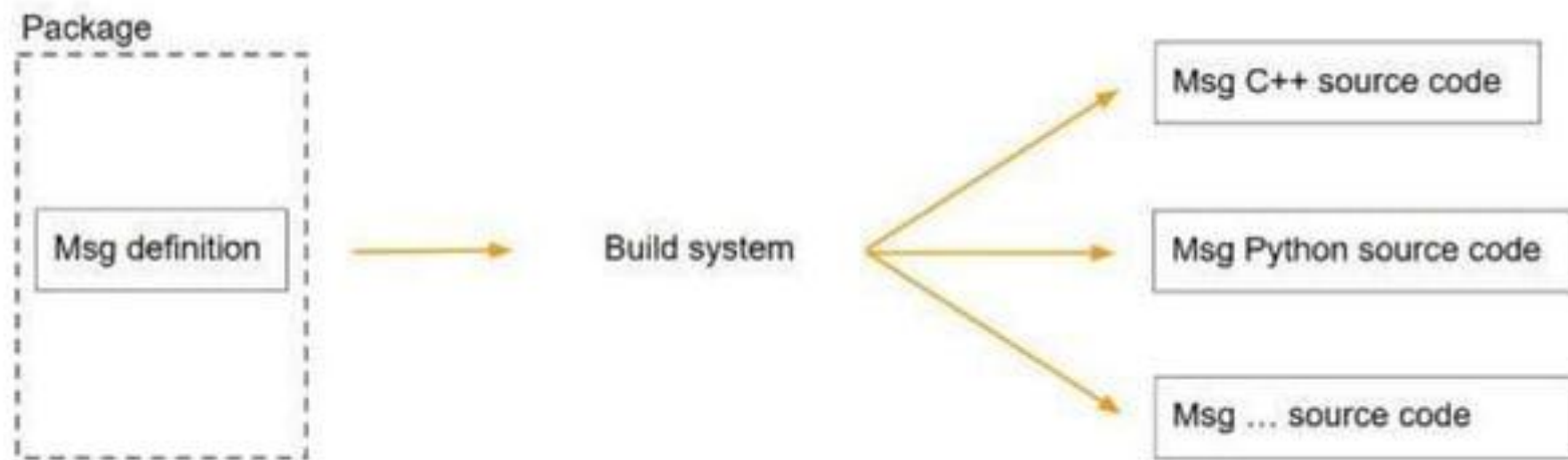
- A topic may be introduced, and various publishers may take turns publishing to that topic.
- **Publishers and subscribers are anonymous.** A publishers only knows it is publishing to a topic, and a subscriber only knows it is subscribing to a topic. Nothing else.
- A topic has a message type. All publishers and subscribers on this topic **must use the message type associated with the topic.**

ROS Messages

- A message is a simple data structure, comprising typed fields. Standard primitive types (integer, floating point, Boolean, etc.) are supported, as are arrays of primitive types. Messages can include arbitrarily nested structures and arrays (much like C structs).
- The messages are basically going through the topic.

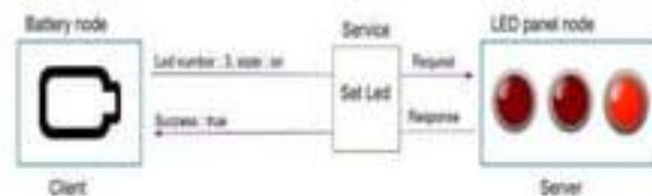
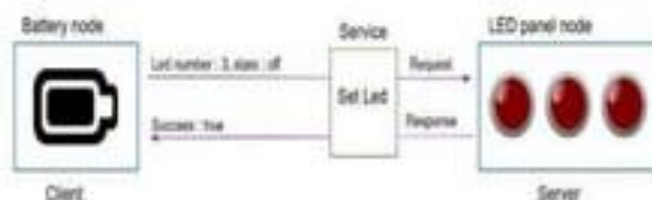


Creating the Messages



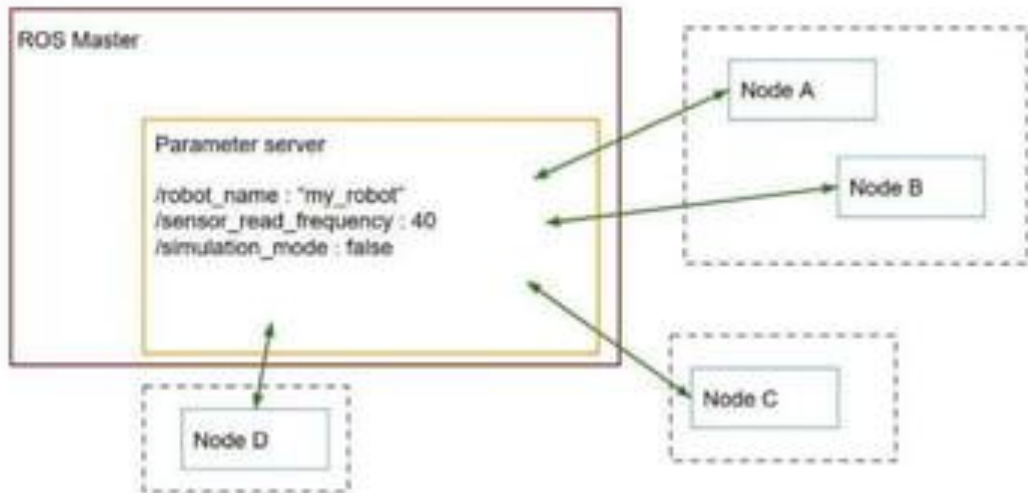
ROS SERVICES

- We have already seen ROS Topics, which is having publishing and subscribing mechanism.
- The ROS Service has Request/Reply mechanism. A service call is a function, which can call whenever a client node sends a request.
- The node who create a service call is called Server node and who call the service is called client node.



ROS Parameters

- *ROS parameter server is a program that normally runs along with the ROS master.*
- The user can store various parameters or values on this server and all the nodes can access it.
- The user can set privacy of the parameter too. If it is a public parameter, all the nodes have access; if it is private, only a specific node can access the parameter.

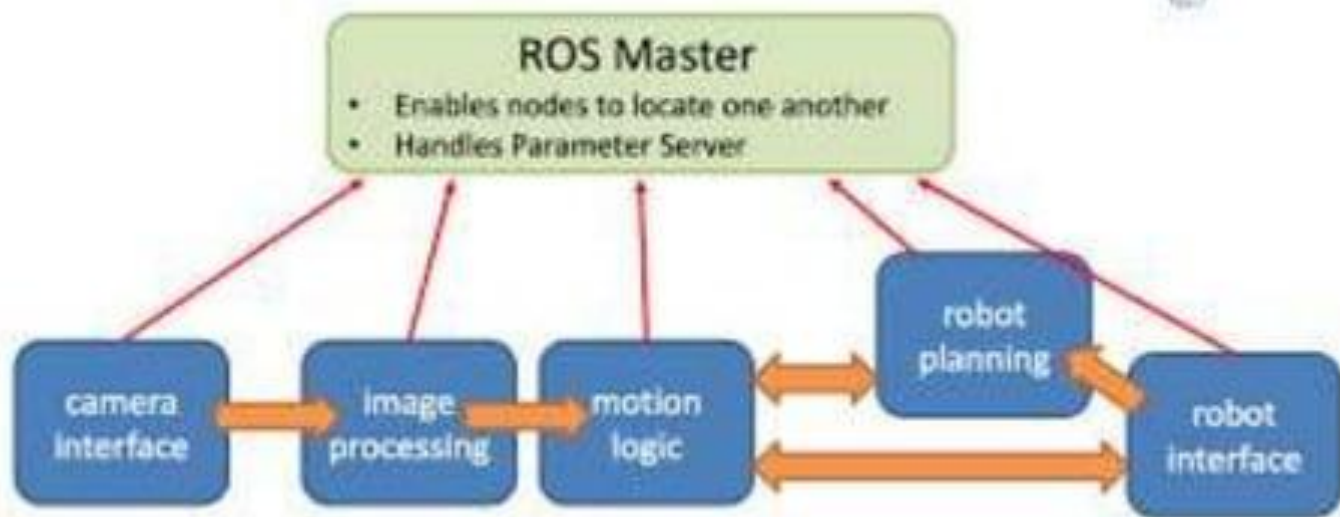


ROS Bags

- Data logging is a vital debugging tool. It is common in ROS systems to log data to file for later analysis and playback. Data logging works as you might expect subscribe to the topic(s) that you want to log, and then write incoming messages to disk.
- In fact, you could easily write your own node to log data for your application. However, you shouldn't write your own logger, because ROS provides a powerful, general logging tool called rosbag. The rosbag tool is able to log data of any type from any ROS topic, all to a single file. By convention, the resulting log files have the extension .bag and are referred to as "ROS bags," or simply, "bags."

ROS Master

- ROS Master: An intermediate program that connects ROS nodes.
- When a node wants to publish something, it will inform the ROS master. When another node wants to subscribe to a topic, it will ask the ROS master from where it can get the data. You can see the **ROS master as a DNS server for nodes** to find where to communicate.



Creating ROS Applications

- **How to create:**

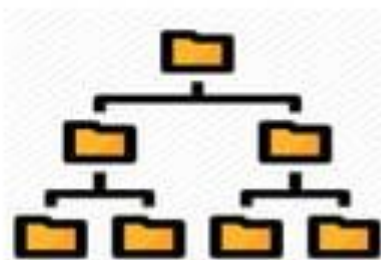
1. ROS workspace.
2. ROS package.
3. ROS nodes.

ROS Build System : CATKIN

- catkin is a build system for compiling ROS packages.
(<http://wiki.ros.org/catkin>).
- catkin is a custom build system made from the CMake build system and Python scripting.

ROS Workspace

ROS Workspace



1. The first step in ROS development is the creation of the ROS workspace, which is where ROS packages are kept. We can create new packages, install existing packages, build and create new executables.
2. A workspace is simply a set of directories in which a related set of ROS code lives.



```
graph TD; A[ROS Workspace] --> B[src folder]; A --> C[build folder]; A --> D[devel folder];
```

ROS Workspace

src folder

build folder

devel folder

src folder

- The *src folder* is the place where you can create, or clone, new packages from repositories.

NOTE: ROS packages only build and create an executable when it is in the src folder

build folder

- The catkin tool creates some build files and intermediate cache CMake files inside the build folder.
- These cache files help prevent from rebuilding all the packages when running the `catkin_make` command.

devel folder

- All the executable files results from building the packages is stored inside the devel folder, which has shell script files to add the current workspace to the ROS workspace path.

Here's A typical workspace layout

```
workspace_folder/
  src/
    CMakeLists.txt
    package_1/
      CMakeLists.txt
      package.xml
    ...
    package_n/
      CMakeLists.txt
      package.xml
    ...
  build/
    CATKIN_IGNORE
  devel/
    bin/
    etc/
```

```
-- WORKSPACE
-- SOURCE SPACE
-- The 'toplevel'
  CMake file
-- BUILD SPACE
-- Keeps catkin from
  walking this directory
-- DEVELOPMENT SPACE
```

So How to create a ROS Workspace?!





```
$ mkdir ~/<workspace_name>/src
```

```
$ cd ~/<workspace_name>/src
```

```
$ catkin_init_workspace
```

```
$ cd ..
```

```
$ catkin_make
```

```
$ source ~/<workspace_name>/devel/setup.bash
```

Let's see it in Action

ROS Packages

What is ROS packages?!



ROS software is organized into packages, each of which contains some combination of code, data, and documentation.



The ROS package is where ROS nodes are organized—libraries and so forth

How to create a ROS package?!

```
$ catkin_create_pkg ros_package_name package_dependencies
```

```
.
├── CMakeLists.txt -> /opt/ros/kinetic/share/
├── hello_world
│   ├── CMakeLists.txt
│   ├── include
│   │   └── hello_world
│   ├── package.xml
│   └── src
```

Note make sure you are at the source file when you creating your packages

So what is the purpose of these files?!

CMakeLists.txt:

- has all the commands to build the ROS source code inside the package and create the executable.

package.xml:

- An XML file mainly contains the package dependencies, information, ..etc.

src:

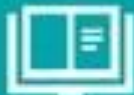
- Contains the source code of ROS package.

include:

- contains the package header files.

ROS Client Libraries

What are ROS Client libraries



The ROS client libraries are a collection of code with functions to implement ROS concepts. We can simply include these library functions in our code to make it a ROS node.



The client library saves development time because it provides the built-in functions to make a ROS application.



Think of it as c++ libraries that you include using the command `#include<library_name.h>` at the beginning of your source code to use c++ functions such cout and cin



Main ROS Client Library

1:

Roscpp:

ROS client library for C++. It is widely used for developing ROS applications because of its high performance.

2:

Rospy:

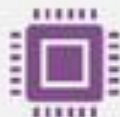
the ROS client library for Python Advantages: Saving development time. ideal for quick prototyping applications Disadvantage: performance is weaker than with roscpp

3:

Roslisp:

the ROS client library of the Lisp language. It is mainly used in motion planning libraries on ROS it is not as popular as roscpp and rosipy.

Creating ROS Nodes



Include the header files and modules used in ROS nodes.



How to initialize a ROS node.



How to publish and to subscribe a topic.

1: Header Files and ROS Modules



C++ → THE FIRST SECTION INCLUDES THE HEADER FILES

Example: To create a ROS C++ node, we have to include the following header files.
`#include "ros/ros.h"`
The **ros.h** has all the headers required to implement ROS functionalities.



PYTHON → THE FIRST SECTION IMPORTS PYTHON MODULES

In Python, we have to import modules to create a ROS node. The ROS module that we need to import is:
`import rospy`

ROS Header files and Modules



- ROS message header:

- `std_msgs`: is a ROS Package that has a message definition of standard data types, such as int, float, string, and so forth.

```
#include "std_msgs/String.h"  
#include "std_msgs/Int32.h"  
#include "std_msgs/Int64.h"
```

- If there is a custom message type, we can call it with the following syntax:

```
#include  
"msg_pkg_name/message_name.h"
```

NOTE: The complete list of message types inside the `std_msgs` package is at

For python:

- **rospy** has all the important ROS functions. To import a message type, we must import the specific modules, like we did in C++.



The following is an example of importing a string message type in Python.

```
from std_msgs.msg import String
```

2: Initializing a ROS Node



- This is a mandatory step in any ROS node.
- In C++, we initialize using the following line of code:

```
int main(int argc, char **argv)
{
    ros::init(argc, argv, "name_of_node")
    .....
}
```

- Meanwhile In Python, we use the following line of code:

```
rospy.init_node('name_of_node',
```

Printing Messages in a ROS Node

- `ROS_INFO(string_msg,args)`: Logging the information of node
- `ROS_WARN(string_msg,args)`: Logging warning of the node
- `ROS_DEBUG(string_msg ,args)`: Logging debug messages
- `ROS_ERROR(string_msg ,args)`: Logging error messages
- `ROS_FATAL(string_msg ,args)`: Logging Fatal messages
- Ex: `ROS_DEBUG("Hello %s","World");`

Creating ROS Node Handler

- After initializing the node, we have to create a NodeHandle instance that starts the ROS node and other operations, like publishing/subscribing a topic. We are using the `ros::NodeHandle` instance to create those operations.
- In C++, the following shows how to create an instance of
`ros::NodeHandle.`
`ros::NodeHandle nh;`
- The rest of the operations in the node use the `nh` instance.
- In Python, we don't need to create a handle; the `rospy` module internally handles it.



Hello World Example

Steps to build the program

- I. Creating the workspace
- II. Creating a hello_world Package
- III. Creating a ROS C++ Node
- IV. Editing the CMakeLists.txt File
- V. Building C++ Nodes
- VI. Executing C++ Nodes

1. Creating the workspace

```
$mkdir -p ~/myHello/src
```

```
$cd ~/myHello/src
```

```
$catkin_init_workspace
```

```
$cd ..
```

```
$catkin_make
```

2. Creating a hello_world Package

```
$ cd ~/myHello/src
```

```
$ myHello/src $ catkin_create_package hello_world roscpp rospy std_msgs
```

```
└─ CMakeLists.txt -> /opt/ros/kinetic/share/  
hello_world  
├─ CMakeLists.txt  
├─ include  
│   └─ hello_world  
├─ package.xml  
└─ src
```

Package.xml

is package.xml. As discussed, this file has information about the package and its dependencies

You can edit this file and add dependencies, package information, and other information to the package

```
<?xml version="1.0"?>
<package>
  <name>hello_world</name>
  <version>0.0.0</version>
  <description>The hello_world package</description>
  <maintainer email="robot@todo.todo">robot</maintainer>
  <license>TODO</license>

  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
  <run_depend>roscpp</run_depend>
  <run_depend>rospy</run_depend>
  <run_depend>std_msgs</run_depend>

  <export>
  </export>
</package>
```

CMakeLists.txt

The `find_package()` finds the necessary dependencies of this package.

If these packages are not available, we can't able to build this package.

The `catkin_package()` is a catkin-provide CMake macro used for specifying catkin-specific information to the build system.

```
cmake_minimum_required(VERSION 2.8.3)
project(hello_world)

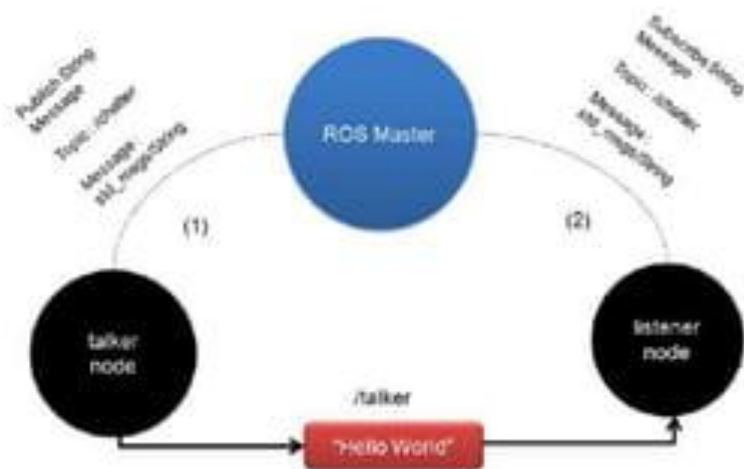
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
)

catkin_package()

include_directories(
  ${catkin_INCLUDE_DIRS}
)
```

3. Creating a ROS C++ Node

- The application can be divided into two nodes.
- The first Node Is the talker node which is responsible of creating the message "Hello World"
- The second node is the listener node which will subscribe to the talker topic and thus receive the messages sent by the talker node through it.



Creating a ROS C++ Node | Publisher Node

- Go to the src node and create a C++ file name it "talker.cpp" with the following code:

```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <sstream>
int main(int argc, char **argv)
{
  ros::init(argc, argv, "talker");
  ros::NodeHandle n;
  ros::Publisher chatter_pub = n.advertise<std_
  msgs::String>("chatter", 1000);
  ros::Rate loop_rate(10);
  int count = 0;
```

```
while (ros::ok())
{
  std_msgs::String msg;
  std::stringstream ss;
  ss << "hello world " << count;
  msg.data = ss.str();
  ROS_INFO("%s", msg.data.c_str());
  chatter_pub.publish(msg);
  ros::spinOnce();
  Chapter 5 Programming with ROS
  193
  loop_rate.sleep();
  ++count;
}
return 0;
}
```


Creating a ROS C++ Node | Subscriber Node

```
• #include "ros/ros.h"
• #include "std_msgs/String.h"
• void chatterCallback(const std_msgs::String::ConstPtr& msg)
• {
•   ROS_INFO("I heard: [%s]", msg->data.c_str());
• }
• int main(int argc, char **argv)
• {
•   ros::init(argc, argv, "listener");
•   ros::NodeHandle n;
•   ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback);
•   ros::spin();
•   return 0;
• }
```

4. Editing the CMakeLists.txt File

- After saving the two files in the hello_world/src folder, the nodes need to be compiled to create the executable. To do this, we have to edit the CMakeLists.txt file, which is not too complicated. We need to add four lines of code to CMakeLists.txt.

```
include_directories(  
  ${catkin_INCLUDE_DIRS}  
)  
  
add_executable(talker src/talker.cpp)  
target_link_libraries(talker  
  ${catkin_LIBRARIES}  
)  
  
add_executable(listener src/listener.cpp)  
target_link_libraries(listener  
  ${catkin_LIBRARIES}  
)
```

5. Building C++ Nodes

After saving CMakeLists.txt, we can build the source code. The command to build the nodes is `catkin_make`. Just switch to the workspace folder and execute the `catkin_make` command. To switch to the `catkin_ws` folder, assume that the workspace is in the home folder:

```
$ cd ~/catkin_ws
```

Executing the `catkin_make` command to build the nodes:

```
$ catkin_make
```

```
Scanning dependencies of target listener
Scanning dependencies of target talker
[ 25%] Building CXX object hello_world/CMakeFiles/listener.dir/src/listener.cpp.o
[ 50%] Building CXX object hello_world/CMakeFiles/talker.dir/src/talker.cpp.o
[ 75%] Linking CXX executable /home/robot/catkin_ws/devel/lib/hello_world/talker
[100%] Linking CXX executable /home/robot/catkin_ws/devel/lib/hello_world/listener
[100%] Built target listener
[100%] Built target talker
robot@robot-pc:~/catkin_ws$
```

6. Executing C++ Nodes

- After building the nodes, the executables are generated inside the `catkin_ws/devel/lib/hello_world` folder



Executing C++ Nodes

- After creating the executable, we can run it on a Linux terminal:

- Starting `roscore`:
- `$ roscore`
- starting the talker node:
- `$ roslaunch hello_world talker`
- The node prints messages on the terminal. Check the list of ROS topics in the system by using the following command:
- `$ rostopic list`
- The listener node can start in another terminal:
- `$ roslaunch hello_world listener`

```
roscore
* /roscore: killed
* /roscore: 1.12.3
roscore
auto-starting new master
process[roscore]: started with pid [4700]
pid_4700, url=http://roscore-113117
waiting /usr/lib/roscore-1.12.3/lib-roscore-113117
process[roscore-1]: started with pid [4800]
started core service [/roscore]
roslaunch hello_world talker
[INFO] [131225487.501203400]: hello world 0
[INFO] [131225487.602802000]: hello world 1
[INFO] [131225487.703776000]: hello world 2
[INFO] [131225487.805488000]: hello world 3
[INFO] [131225487.906386000]: hello world 4
[INFO] [131225488.007912000]: hello world 5
[INFO] [131225488.109426000]: hello world 6
[INFO] [131225488.210841000]: hello world 7
[INFO] [131225488.312354000]: hello world 8
[INFO] [131225488.413868000]: hello world 9
[INFO] [131225488.515382000]: hello world 10
[INFO] [131225488.616896000]: hello world 11
[INFO] [131225488.718410000]: hello world 12
roslaunch hello_world listener
[INFO] [131225487.501203400]: heard: hello world 0
[INFO] [131225487.602802000]: heard: hello world 1
[INFO] [131225487.703776000]: heard: hello world 2
[INFO] [131225487.805488000]: heard: hello world 3
[INFO] [131225487.906386000]: heard: hello world 4
[INFO] [131225488.007912000]: heard: hello world 5
[INFO] [131225488.109426000]: heard: hello world 6
[INFO] [131225488.210841000]: heard: hello world 7
[INFO] [131225488.312354000]: heard: hello world 8
[INFO] [131225488.413868000]: heard: hello world 9
[INFO] [131225488.515382000]: heard: hello world 10
[INFO] [131225488.616896000]: heard: hello world 11
[INFO] [131225488.718410000]: heard: hello world 12
```


Resources to learn ROS

Useful Books



The ROS Community

- • The *ROS distribution* is a collection of versioned packages.
- • The *ROS wiki* has tutorials on how to set up and program ROS.
- • *ROS Answers* (<https://answers.ros.org/questions/>) has ROS queries and solutions, similar to Stack Overflow.
- • *ROS discourse* (<https://discourse.ros.org>) is a forum in which developers can share news and ask queries related to ROS.